Integration Manual

for MPC560XB FEE Driver

Document Number: IM13FEEASR4.0 Rev0003R1.0.1
Rev 1.2

freescale™

Contents

Section number **Title** **Page**

Chapter 1
Revision History

Table 1-1. Revision History

Revision	Date	Author	Description
1.0	15-Feb-2012	Gaetano Stabile	Initial version
1.1	15-May-2013	Rakesh Ranjan	Updated for Bolero RTM 1.0.0
1.2	28-May-2014	Nicolas Regent	Updated for Bolero RTM 1.0.1

Chapter 2
Introduction

This integration manual describes the integration requirements for FEE Driver for MPC560XB microcontrollers.

AUTOSAR FEE driver requirements and APIs are described in the AUTOSAR 4.0 Rev0003FEE Driver Software Specification Document (version V4.0 Rev003)
[Reference List]

The roadmap for the document is as follows:

[Build Options] This section gives a brief overview of the build procedure (compiler, linker options and source files) and EB tresos Studio plugin setup.

[Function Calls during Start-up] This section lists the various function calls to modules during Start-up, Shutdown and Wake-up.

[Exclusive areas to be defined in BSW scheduler]This section specifies the various module requirements related to:

- Exclusive areas to be defined in SchM module
- Peripheral Hardware Requirements
- ISR to configure within OS
- Specific interface to other modules
- Dependencies with other modules

[Main functions calls within BSW scheduler] This section specifies the requirements related to to the main API and gives a brief overview of the main functions calls within SchM module, and calls to notification functions, callbacks, callouts.

[Sections to be defined in MemMap.h] This section describes the memory allocation requirements namely the sections to be defined in MemMap.h file and the linker command file.

[Configuration Parameters] This section lists the FEE module configuration parameters and their variants/classes.

[Files required for Compilation] This section describes in brief the steps for integrating FEE module.

2.1 Supported Derivatives

The software described in this document is intented to be used with the following microcontroller devices of Freescale Semiconductor.

Table 2-1. MPC5607B Derivatives

Freescale Semiconductor	mpc5601dxlh_lqfp64 mpc5601dxll_lqfp100 mpc5602dxlh_lqfp64 mpc5602dxll_lqfp100 mpc5602bxlh_lqfp64 mpc5602bxll_lqfp100 mpc5602bxlq_lqfp144 mpc5602cxlh_lqfp64 mpc5602cxll_lqfp100 mpc5603bxlh_lqfp64 mpc5603bxll_lqfp100 mpc5603bxlq_lqfp144 mpc5603cxlh_lqfp64 mpc5603cxll_lqfp100 mpc5604bxlh_lqfp64 mpc5604bxll_lqfp100 mpc5604bxlq_lqfp144 mpc5604bxmg_mapbga208 mpc5604cxlh_lqfp64 mpc5604cxll_lqfp100 mpc5605bxll_lqfp100 mpc5605bxlq_lqfp144 mpc5605bxlu_lqfp176 mpc5606bkll_lqfp100 mpc5606bxlq_lqfp144 mpc5606bxlu_lqfp176 mpc5607bxlu_lqfp176 mpc5607bxmg_mapbga208

All of the above microcontroller devices are collectively named as MPC560XB.

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.

- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About this Manual

This Technical Reference employs the following typographical conventions:

Boldface type: Bold is used for important terms, notes and warnings.

Italic font: Italic typeface is used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

2.4 Acronyms and Definitions

Table 2-2. Acronyms and Definitions

Term	Definition
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
DEM	Diagnostic Event Manager
DET	Development Error Tracer
ECU	Electronic Control Unit
EEPROM	Electrically Erasable Programmable Read-Only Memory
FEE	Flash EEPROM Emulation Module/Driver
FLS	Flash Driver
MCU	Micro Controller Unit
VLE	Variable Length Encoding
XML	Extensible Markup Language
ISR	Interrupt Service Routine
OS	Operating System

Table continues on the next page...

Table 2-2. Acronyms and Definitions (continued)

Term	Definition
VSDM	Vendor-Specific Module Definition
CER	Catastrophic Error Recovery
SchM	Schedule Manager
MemIf	Memory Interface Module
EcuM	ECU Manager Module
BSW	Basic Software Module

2.5 Reference List

Table 2-3. Reference List

#	Title	Version
1	AUTOSAR 4.0 Rev0003FEE Driver Software Specification Document.	V4.0 Rev003
2	MPC5607B Reference Manual	Rev. 7.2, May 2012
3	MPC5604B Reference Manual	Rev. 8.2, September 2013
4	5606B Reference Manual	Rev. 2, May 2014
5	5602D Reference Manual	Rev. 4.2, Dec 2013
6	Bolero 1.5M cut 2.2 Errata Sheet Reference Manual	MPC5607B_1M03Y - Rev. 12Dec2013
7	Bolero 512k cut 2.3 Errata Sheet Reference Manual	MPC5604B_2M27V - Rev. 26 FEB 2014
8	Bolero 256k cut 1.1 Errata Sheet Reference Manual	MPC5602D_1M18Y - Rev. , 12 DEC 2013

Chapter 3
Building the Driver

This section describes the source files and various compilers, linker options used for building the Autosar FEE driver for Freescale SemiconductorMPC560XB . It also explains the EB Tresos Studio plugin setup procedure.

3.1 Build Options

The FEE driver files are compiled using:

- Green Hills MULTI v6.1.4
- Windriver DIAB Rel DIAB_5_9_2_0-FCS_20120725_234511
- Freescale CodeWarrior for MPC5xxx v2.10 - compiler Version 4.3 build 224
- HighTec v4.6.4.0, Compiler gcc v4.6.3

The compiler and linker flags used for building the driver are explained below.

Note

The TS_T2D13M10I1R0 plugin name is composed as follow:

TS_T = Target_Id

D = Derivative_Id

M = SW_Version_Major

I = SW_Version_Minor

R = Revision

(i.e. Target_Id = 2 identifies PowerPC architecture and
Derivative_Id = 13 identifies the MPC560XB)

3.1.1 CW Compiler/Linker/Assembler Options

Table 3-1. Compiler Options

Option	Description
-proc Zen	Generates and links object code for Zen processor. The compiler uses unsigned as the default parameter for the -char switch
-lang c	Expects source code to conform to the language specified by the ISO/IEC 9899-1990 ("C90") standard
-opt all	This option is selected all optimization (the same as -opt speed,level=4,intrinsics,noframe)
-common off	Disables moving uninitialized data into a common section
-sdatathreshold 0	Specifies the threshold size (in bytes) for an item considered by the linker to be small data. (The linker stores small data items in the Small Data address space. The compiler can generate faster code to access this data.)
-sdata2threshold 0	Specifies the threshold size (in bytes) for an item considered by the linker to be small constant data. (The linker stores small constant data items in the Small Constant Data address space.)
-vle	Tells the compiler and linker to generate and lay out Variable Length Encoded (VLE) instructions, available on Zen variants of Power Architecture processors
-use_lmw_stmw on	Enables the use of multiple load and store instructions for function prologues and epilogues
-ppc_asm_to_vle	Converts regular Power Architecture assembler mnemonics to equivalent VLE (Variable Length Encoded) assembler mnemonics in the inline assembler
-Cpp_exceptions off	When on, generates executable code for C++ exceptions. When off, generates smaller, faster executable code
-func_align 4	Specifies alignment of functions in executable code
-sym dwarf-2,full	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format)
-gdwarf-2	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format). The linker ignores debugging information that is not in the Dwarf 1, Dwarf 2 format
-w on	Turns on most warning messages
-r	Compiler should expect function prototypes
-w undefmacro	Issues warning messages on the use of undefined macros in #if and #elif conditionals
-char unsigned	Controls the default sign of the char data type: char data items are unsigned
-nosyspath	Performs a search of both the user and system paths, treating #include statements of the form #include xyz the same as the form #include "xyz"
-fp none	No floating point code generation
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DMWERKS	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the CW preprocessor symbol.

Table 3-2. Assembler Options

Option	Description
-proc Zen	Generates and links object code for Zen processor. The compiler uses unsigned as the default parameter for the -char switch

Table continues on the next page...

Table 3-2. Assembler Options (continued)

Option	Description
-vle	Tells the compiler and linker to generate and lay out Variable Length Encoded (VLE) instructions, available on Zen variants of Power Architecture processors
-sym dwarf-2,full	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format)
-gdwarf-2	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format). The linker ignores debugging information that is not in the Dwarf 1, Dwarf 2 format.
-DMWERKS	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the CW preprocessor symbol.

Table 3-3. Linker Options

Option	Description
-proc Zen	Generates and links object code for Zen processor. The compiler uses unsigned as the default parameter for the -char switch
-code_merging all	Removes duplicated functions to reduce object code size
-far_near_addressing	Simplifies address computations to reduce object code size and improve performance
-vle_enhance_merging	Removes duplicated functions that are called by functions that use VLE instructions to reduce object code size
-listdwarf	DWARF debugging information in the linker's map file
-sym dwarf-2,full	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format)
-char unsigned	Controls the default sign of the char data type: char data items are unsigned.

3.1.2 DIAB Compiler/Linker/Assembler Options

Table 3-4. Compiler Options

Option	Description
-Xdialect-ansi	Follow the ANSI C standard with some additions
-XO	Enables extra optimizations to produce highly optimized code
-g	Generate symbolic debugger information and disable some optimizations.
-Xsize-opt	Optimize for size rather than speed when there is a choice
-Xsmall-data=0	Set Size Limit for "small data" Variables to zero.
-Xaddr-sconst=0x11	Specify addressing for constant static and global variables with size less than or equal to -Xsmall-const to far-absolute.
-Xaddr-sdata=0x11	Specify addressing for non-constant static and global variables with size less than or equal to -Xsmall-data in size to far-absolute.
-Xno-common	Disable use of the "COMMON" feature so that the compiler or assembler will allocate each uninitialized public variable in the .bss section for the module defining it, and the linker will require exactly one definition of each public variable
-Xnested-interrupts	Allow nested interrupts

Table continues on the next page...

Table 3-4. Compiler Options (continued)

Option	Description
-Xdebug-dwarf2	Generate symbolic debug information in dwarf2 format
-Xdebug-local-all	Force generation of type information for all local variables
-Xdebug-local-cie	Create common information entry per module
-Xdebug-struct-all	Force generation of type information for all typedefs, struct, union and class types
-Xforce-declarations	Generates warnings if a function is used without a previous declaration
-ee1481	Generate an error when the function was used before it has been declared
-Xmacro-undefined-warn	Generates a warning when an undefined macro name occurs in a #if preprocessor directive
-W:as:,-l	Pass the option "-l" (lower case letter L) to the assembler to get an assembler listing file
-Wa,-Xisa-vle	Instruct the assembler to expect and assemble VLE (Variable Length Encoding) instructions rather than BookE instructions.
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DDIAB	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the DIAB preprocessor symbol.
-Xforce-prototypes	-Generate warnings if a function is used without a previous prototype declaration.
-tPPCE200Z0VEN:simple	Sets target processor to PPCE200Z0, generates ELF using EABI conventions, No floating point support (minimizes the required runtime), selects simple environment settings for Startup Module and Libraries.

Table 3-5. Assembler Options

Option	Description
-tPPCE200Z0VEN:simple	Sets target processor to PPCE200Z0, generates ELF using EABI conventions, No floating point support (minimizes the required runtime), selects simple environment settings for Startup Module and Libraries
-g	Dump the symbols in the global symbol table in each archive file.
-Xisa-vle	Expect and assemble VLE (Variable Length Encoding) instructions rather than Book E instructions. The default code section is named .text_vle instead of .text, and the default code section fill "character" is set to 0x44444444 instead of 0. The .text_vle code section will have ELF section header flags marking it as VLE code, not Book E code.

Table 3-6. Linker Options

Option	Description
-tPPCE200Z0VEN:simple	Sets target processor to PPCE200Z0, generates ELF using EABI conventions, No floating point support (minimizes the required runtime), selects simple environment settings for Startup Module and Libraries
-Xelf	Generates ELF object format for output file
-m6	Generates a detailed link map and cross reference table
-lc	Specifies to linker to search for libc.a
-Xlibc-old	Enables usage of legacy (pre-release 5.6) libraries

3.1.3 GHS Compiler/Linker/Assembler Options

Table 3-7. Compiler Options

Option	Description
-cpu=ppc560xb	Selects target processor: ppc560xb
-ansi	Specifies ANSI C with extensions. This mode extends the ANSI X3.159-1989 standard with certain useful and compatible constructs.
-noSPE	Disables the use of SPE and vector floating point instructions by the compiler.
-Ospace	Optimize for size
-sda=0	Enables the Small Data Area optimization with a threshold of 0.
-vle	Enables VLE code generation
-dual_debug	Enables the generation of DWARF, COFF, or BSD debugging information in the object file
-G	Generates source level debugging information and allows procedure call from debugger's command line.
--no_exceptions	Disables support for exception handling
-Wundef	Generates warnings for undefined symbols in preprocessor expressions
-Wimplicit-int	Issues a warning if the return type of a function is not declared before it is called
-Wshadow	Issues a warning if the declaration of a local variable shadows the declaration of a variable of the same name declared at the global scope, or at an outer scope
-Wtrigraphs	Issues a warning for any use of trigraphs
--prototype_errors	Generates errors when functions referenced or called have no prototype
--incorrect_pragma_warnings	Valid #pragma directives with wrong syntax are treated as warnings
-noslashcomment	C++ like comments will generate a compilation error
-preprocess_assembly_files	Preprocesses assembly files
--short_enum	Store enumerations in the smallest possible type
--no_commons	Allocates uninitialized global variables to a section and initializes them to zero at program startup.
-c	Produces an object file (called input-file.o) for each source file.
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DUSE_SW_VECTOR_MODE	-D defines a preprocessor symbol and optionally can set it to a value. USE_SW_VECTOR_MODE: By default in the package, drivers are compiled to be used with interrupt controller configured to be in hardware vector mode. In case of AUTOSAR_OS_NOT_USED, the compiler option "-DUSE_SW_VECTOR_MODE" must be added to the list of compiler options to be used with interrupt controller configured to be in software vector mode.
-DDISABLE_MCAL_INTERMODULE_ASR_CHECK	-D defines a preprocessor symbol to disable the inter-module version check for AR_RELEASE versions. DISABLE_MCAL_INTERMODULE_ASR_CHECK: By default in the package, drivers are compiled to perform the inter-module version check as per Autosar BSW004. When the inter-module version check needs to be disabled then the DISABLE_MCAL_INTERMODULE_ASR_CHECK global define must be added to the list of compiler options.
-DGHS	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the GHS preprocessor symbol.

Table 3-8. Assembler Options

Option	Description
-cpu=ppc560xb	Selects target processor: ppc560xb

Table 3-9. Linker Options

Option	Description
-cpu=ppc560xb	Selects target processor: ppc560xb
-nostartfiles	Do not use Start files.
-vle	Enables VLE code generation
-linker_warnings	Display linker warnings

3.1.4 HT Compiler/Linker/Assembler Options

Table 3-10. Compiler Options

Option	Description
-Os	Optimize for size. '-Os' enables all '-O2' optimizations that do not typically increase code size. It also performs further optimizations designed to reduce code size.
-msdata=eabi	On System V.4 and embedded PowerPC systems, put small initialized const global and static data in the '.sdata2' section, which is pointed to by register r2. Put small initialized non-const global and static data in the '.sdata' section, which is pointed to by register r13. Put small uninitialized global and static data in the '.sbss' section, which is adjacent to the '.sdata' section.
-G1000	On embedded PowerPC systems, put global and static items less than or equal to num bytes into the small data or bss sections instead of the normal data or bss section.
-fshort-double	Use the same size for double as for float.
-mno-spe	This option used to disable spe extension (Like –noSPE on ghs compiler).
-mregnames	On System V.4 and embedded PowerPC systems do (do not) emit register names in the assembly language output using symbolic forms
-g3	Request debugging information of Level 3. Includes extra information, such as all the macro definitions present in the program.
-Wall	This enables all the warnings about constructions that some users consider questionable, and that are easy to avoid (or modify to prevent the warning), even in conjunction with macros.
-mvle	Enable support of VLE
--mcpu=z425n3	Support of e200 Z425n3 core
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DUSE_SW_VECTOR_MODE	-D defines a preprocessor symbol and optionally can set it to a value. USE_SW_VECTOR_MODE: By default in the package, drivers are compiled to be used with interrupt controller configured to be in hardware vector mode. In case of AUTOSAR_OS_NOT_USED, the compiler option "-DUSE_SW_VECTOR_MODE" must be

Table 3-10. Compiler Options

Option	Description
	added to the list of compiler options to be used with interrupt controller configured to be in software vector mode.

Table 3-11. Assembler Options

Option	Description
-c	Compile or assemble the source files, but do not link. The linking stage simply is not done. The ultimate output is in the form of an object file for each source file.
-Wa,--gdwarf2	Produce debugging information in DWARF format.
-msdata=eabi	Put small initialized const global and static data in the .sdata2 section, which is pointed to by register r2. Put small initialized non-const global and static data in the .sdata section, which is pointed to by register r13. Put small uninitialized global and static data in the .sbss section, which is adjacent to the .sdata section.
-mregnames	Do emit register names in the assembly language output using symbolic forms.
-mno-spe	This option used to disable spe extension (Like –noSPE on ghs compiler).
-mcpu=z425n3	Support of e200 z425n3 core

Table 3-12. Linker Options

Option	Description
-N	Set the text and data sections to be readable and writable. Also, do not page-align the data segment. If the output format supports Unix style magic numbers, mark the output as OMAGIC.
--no-undefined	Normally when a symbol has an unde ned version, the linker will ignore it. This option disallows symbols with unde ned version and a fatal error will be issued instead.
--warn-common	Normally ppc-vle-ld will give a warning if it nds an incompatible library during a library search. This option silences the warning.
--error-unresolved-symbols	This restores the linker's default behaviour of generating errors when it is reporting unresolved symbols.
-nocrt0	library crt0 from compiler library not used, it uses custom crt0 file
-nostartfiles	no start file from compiler library is used
-Wl	If the linker is being invoked indirectly, via a compiler driver (e.g. ‘gcc’) then all the linker command line options should be prefixed by ‘-Wl,’Used in combination with -no-warn-flags, --no-undefined, --warn-common and --error-unresolved-symbols
--no-warn-flags	Used in combination with -Wl disable linker warnings. See previous
-T	The -T option specifies the name of your own script le sprig.link. The -c is a synonym of the -T option

3.2 Files required for Compilation

This section describes the include files required to compile, assemble (if assembler code) and link the FEE driver for MPC560XB microcontrollers.

To avoid integration of incompatible files, all the include files from other modules shall have the same AR_MAJOR_VERSION and AR_MINOR_VERSION, i.e. only files with the same AUTOSAR major and minor versions can be compiled.

FEE Files

- ..\Fee_TS_T2D13M10I1R0\src\Fee.c
- ..\Fee_TS_T2D13M10I1R0\include\Fee_Cbk.h
- ..\Fee_TS_T2D13M10I1R0\include\Fee.h
- ..\Fee_TS_T2D13M10I1R0\include\Fee_Api.h
- ..\Fee_TS_T2D13M10I1R0\include\Fee_InternalTypes.h
- ..\Fee_TS_T2D13M10I1R0\include\Fee_Types.h
- ..\Fee_TS_T2D13M10I1R0\include\Fee_Version.h

- Fee_Cfg.h - this file should be generated by the user using a configuration/generation tool
- Fee_Cfg.c - this file should be generated by the user using a configuration/generation tool

Other includes files:

Files from MemIf folder:

- ..\MemIf_TS_T2D13M10I1R0\include\MemIf_Types.h

Files from Base common folder

- ..\Base_TS_T2D13M10I1R0\include\Compiler.h
- ..\Base_TS_T2D13M10I1R0\include\Compiler_Cfg.h
- ..\Base_TS_T2D13M10I1R0\include\MemMap.h
- ..\Base_TS_T2D13M10I1R0\include\Platform_Types.h
- ..\Base_TS_T2D13M10I1R0\include\Mcal.h
- ..\Base_TS_T2D13M10I1R0\include\Std_Types.h

Files from Det folder:

- ..\Det_TS_T2D13M10I1R0\include\Det.h

Files from Fls folder:

- ..\Fls_TS_T2D13M10I1R0\include\Fls.h

3.3 Setting up the Plug-ins

The FEE driver was designed to be configured by using the EB Tresos Studio (version Elektrobit Tresos Release 14.2.1 (b140128-1223) or later.)

- VSMD (Vendor Specific Module Definition) file in EB tresos Studio XDM format: .. \Fee_TS_T2D13M10I1R0\config\Fee.xdm
- VSMD (Vendor Specific Module Definition) file in AUTOSAR compliant EPD format: ..\Fee_TS_T2D13M10I1R0\autosar\Fee.epd
- Code Generation Templates for Pre compile time configuration parameters:
  - ..\Fee_TS_T2D13M10I1R0 \generate\include\Fee_Cfg.h
  - ..\Fee_TS_T2D13M10I1R0 \generate\src\Fee_Cfg.c

Steps to generate the configuration:

1. Copy the module folders Fee_TS_T2D13M10I1R0, Fls_TS_T2D13M10I1R0, Base_TS_T2D13M10I1R0 , Resource_ TS_T2D13M10I1R0, into the Tresos plugins folder.
2. Set the desired Tresos Output location folder for the generated sources and header files.
3. Use the EB tresos Studio GUI to modify ECU configuration parameters values.
4. Generate the configuration files.

Chapter 4
Function calls to module

None.

4.1 Function Calls during Start-up

FEE shall be initialized during STARTUP phase of EcuM initialization. The API to be called for this is Fee _Init(). The MCU module should be initialized before the FEE is initialized.

Notes:

- FEE module is the upper layer module which works on FLS module
- Fls_Init API must be called before calling Fee_Init API.
- Fls_MainFunction API and Fee_MainFunction API must be called periodically for FEE module initialization (after Fee_Init the: Fls_MainFunction and Fee_MainFunction must be called until Fee_GetStatus does not return MEMIF_IDLE) and its operation.

4.2 Function Calls during Shutdown

None.

4.3 Function Calls during Wake-up

None.

Chapter 5
Module requirements

None.

5.1 Exclusive areas to be defined in BSW scheduler

None.

5.2 Peripheral Hardware Requirements

The FEE module is hardware independent module and depends on the underlying FLS module and its configuration only.

5.3 ISR to configure within OS – dependencies

None.

5.4 ISR Macro

None.

5.5 Other AUTOSAR Modules Dependencies

- **Base:** This module provides basic data types and auxiliary macros or functions commonly used by other AUTOSAR modules.
- **Dem:** This module is necessary for reporting of production relevant error status. The API function used is Dem_ReportErrorStatus.

- **Det:** This module is necessary for development error tracking. The API function used is Det_ReportError. The activation/deactivation of the Development Error Tracker is configurable using FeeDevErrorDetect configuration parameter.
- **MemIf:** Memory Interface
- **Resource:** Sub-derivative model is selected from Resource configuration.
- **Flash:** The flash driver provides services for reading, writing and erasing flash memory and a configuration interface for modifying the write/erase protection if supported by the underlying hardware.

Chapter 6
Main API Requirements

6.1 Main functions calls within BSW scheduler

None.

6.2 API Requirements

Before calling the Fee_Write function for immediate data, the function Fee_EraseImmediateBlock must be called to pre-erase the flash area.

6.3 Calls to Notification Functions, Callbacks, Callouts

The FEE module provides user-configurable notifications

- FeeNvMJobEndNotification,
- FeeNvMJobErrorNotification.

These are usually routed to the NvM.

Additionally, the FEE module publishes two APIs

- Fee_JobEndNotification,
- Fee_JobErrorNotification,

that must be linked to the appropriate notifications of the FLS module.

6.4 Tips for FEE Integration

Task scheduling

Make sure that no FEE/FLS functions are interrupted by another FEE/FLS function except Fee_Cancel.

Example 1: Fee_MainFunction is called from 10 ms OS task and Fee_Write function is called from 20 ms OS task. It has to be ensured that Fee_Write function is not interrupted by Fee_MainFunction.

Example2: Fee_MainFunction is called from several places in the application. It has to be ensured that Fee_MainFunction is not interrupted by another Fee_MainFunction.

Time consumption

1. Be aware of the maximum time consumption of the FEE/FLS functions (the best is to use actual configuration for performance analysis).

   Example 1: If Fee_MainFunction execution takes up to 5 ms (e.g. when a cluster swap occurs), it is risky to call it from an 1 ms task.

   Example 2: Write operation takes 5 ms to complete. If a cluster swap occurs during the write operation, it may take 500 ms.

2. Be aware of the maximum number of FEE/FLS main function calls or overall time needed for operations (read, write, …) to finish. Again, the best is to use actual configuration for performance analysis.

   Example 1: If "Fls erase blank check" is enabled, 64 KiB clusters are used and "Max erase blank check" is set to 256 bytes, it may take up to 600 FEE/FLS main function calls to complete the swap operation. I.e. if the main functions are called within a 10 ms task, it takes up to 6 s to finish the swap operation.

   Note: presented timing numbers are just an example, please refer to the profiling report to get real numbers.

FEE Block Always Available

According to the AUTOSAR FEE153 and FEE154 requirement, when the write operation starts, corresponding block is marked as inconsistent. It is marked as valid after successful write. It means that if the write operation is interrupted (cancelled, by device reset, power down ...), application cannot access the block data anymore. There is a configuration parameter (FEE Block Always Available) which allows to set the module behavior against this FEE153 and FEE154 requirement. As a result, the FEE will always provide the last valid information. Valid information means FEE_BLOCK_VALID (block is valid) or FEE_BLOCK_INVALID (block was invalidated or block is not present in the cluster at all).

Example 1: Driver behavior is set according FEE153 and FEE154. One instance of the block is successfully written to the memory. Another write operation of this block is scheduled but it is interrupted before finish (by Fee_Cancel or power down). Block can be valid containing old data (operation was interrupted before write process started), block can be valid containing new data (operation was interrupted just before returning success information), or block can be invalid/inconsistent (ongoing write process was interrupted) .

Example 2: Driver behavior is set to violate FEE153 and FEE154 (not supported by all versions). One instance of the block is successfully written to the memory. Another write operation of this block is scheduled but it is interrupted before finish (by Fee_Cancel or power down). Previous instance of the block is still accessible.

Immediate data usage

Immediate data are used for fast write operations, because no swap operation can occur during write (when used properly). The following sequence shall be used for the immediate data usage:

1. Fee_EraseImmediateBlock – to allocate space for the block in advance. In this case a swap operation can occur.
2. Fee_Cancel – to interrupt any internal processing in case the driver is not idle.
3. Fee_Write of immediate block – no swap can occur because space is already allocated.

After the immediate block is written, new space shall be allocated for further usage of this block (Fee_EraseImmediateBlock function). Note that after Fee_EraseImmediateBlock function is called, new space is allocated and the previous instance of the immediate block is no longer accessible.

Example of usage 1:

- *FEE/FLS initialization.*
- *Space reservation for immediate blocks (Fee_EraseImmediateBlock).*
- *Execution of the application code.*
- *Abnormal situation occurs.*
- *All fee operations are stopped (Fee_Cancel).*
- *Immediate blocks are written.*
- *Power down/reset…*
- *FEE/FLS initialization.*
- *Check if immediate data are written. If so, some abnormal situation occurred. The stored information can be used.*
- *Space reservation for immediate blocks – all previous data of these blocks are lost.*

Example of usage 2:

- *Space reservation is done using flash image.*
- *App is running and writing immediate data in case of some special situation (each immediate block is written just once).*
- *When all immediate blocks are written, unit is not writing any immediate data anymore and report error state.*

Code flash erase

Be aware of the read-while-write support in the device when erasing the code flash from code flash. If it is not supported, the access code shall be loaded into the RAM (the "Fls Load Access Code On Job Start" parameter must be turned on). In this case the write/erase operation shall be set as synchronous (if asynchronous write/erase is enabled, access code in the RAM is ignored).

Meaning of the FEE block states (result of the FEE GetJobResult operation)

Table 6-1. Fee Job Information

MemIf_JobResultType	Non-immediate block	Immediate block
MEMIF_JOB_OK	block is valid	block is valid
MEMIF_BLOCK_INVALID	block was invalidated, or block is not in the cluster	block was invalidated, or block is not in the cluster
MEMIF_BLOCK_INCONSISTENT	block has corrupted header (wrong checksum, address, ...), or block doesn't match the configuration (length, immediate, ...)	block has corrupted header (wrong checksum, address, ...), or block doesn't match the configuration (length, immediate, ...), or space for the block is reserved

Chapter 7
Memory Allocation

7.1 Sections to be defined in MemMap.h

For precompile:

```
#ifdef FEE_START_CONFIG_DATA_UNSPECIFIED
#undef FEE_START_CONFIG_DATA_UNSPECIFIED
#undef MEMMAP_ERROR
/*no definition -> default compiler settings are used */
#endif
#ifdef FEE_STOP_CONFIG_DATA_UNSPECIFIED
#undef FEE_STOP_CONFIG_DATA_UNSPECIFIED
#undef MEMMAP_ERROR
/*no definition -> default compiler settings are used */
#endif
```

For code:

```
#ifdef FEE_START_SEC_CODE
#undef FEE_START_SEC_CODE
#undef MEMMAP_ERROR
/*no definition -> default compiler settings are used */
#endif
#ifdef FEE_STOP_SEC_CODE
#undef FEE_STOP_SEC_CODE
#undef MEMMAP_ERROR
/*no definition -> default compiler settings are used */
#endif
```

For variables:

```
#ifdef FEE_START_SEC_VAR_UNSPECIFIED
#undef FEE_START_SEC_VAR_UNSPECIFIED
#undef MEMMAP_ERROR
/*no definition -> default compiler settings are used */
#endif
#ifdef FEE_STOP_SEC_VAR_UNSPECIFIED
#undef FEE_STOP_SEC_VAR_UNSPECIFIED
#undef MEMMAP_ERROR
/*no definition -> default compiler settings are used */
```

```
#endif
```

For constant data:

```
#ifdef FEE_START_SEC_CONST_UNSPECIFIED
#undef FEE_START_SEC_CONST_UNSPECIFIED
#undef MEMMAP_ERROR
/*no definition -> default compiler settings are used */
#endif
#ifdef FEE_STOP_SEC_CONST_UNSPECIFIED
#undef FEE_STOP_SEC_CONST_UNSPECIFIED
#undef MEMMAP_ERROR
/*no definition -> default compiler settings are used */
#endif
```

7.2 Linker command file

Memory shall be allocated for every section defined in MemMap.h.

Chapter 8
Configuration parameters considerations

Configuration parameter class for Autosar FEE driver fall into the following variants as defined below:

8.1 Configuration Parameters

Configuration parameter class for Autosar Fee driver fall into the following variants as defined below:

Table 8-1. Configuration Parameters

Configuration Container	Configuration Parameters	Configuration Variant	Current Implementation
FeeGeneral			
	FeeDevErrorDetect	VariantPreCompile	PreCompile
	FeeIndex	VariantPreCompile	PreCompile
	FeeNvmJobEndNotification	VariantPreCompile	PreCompile
	FeeNvmJobErrorNotification	VariantPreCompile	PreCompile
	FeePollingMode	VariantPreCompile	PreCompile
	FeeVersionInfoApi	VariantPreCompile	PreCompile
	FeeVirtualPageSize	VariantPreCompile	PreCompile
	FeeDataBufferSize	VariantPreCompile	PreCompile
	FeeBlockAlwaysAvailable	VariantPreCompile	PreCompile
FeeSector			
	FeeSectorRef	VariantPreCompile	PreCompile
FeeBlockConfiguration			
	FeeClusterGroupRef	VariantPreCompile	PreCompile
	FeeBlockNumber	VariantPreCompile	PreCompile
	FeeBlockSize	VariantPreCompile	PreCompile
	FeeImmediateData	VariantPreCompile	PreCompile
	FeeNumberOfWriteCycles	VariantPreCompile	PreCompile
	FeeDeviceIndex	VariantPreCompile	PreCompile

Chapter 9
Integration Steps

This section gives a brief overview of the steps needed for integrating FEE Driver :

- Generate the required FEE configurations. For more details refer to Setting up the Plug-ins
- Allocate proper memory sections in MemMap.h and linker command file. For more details refer to section Sections to be defined in MemMap.h
- Make sure all include files for compilation. For more details refer to section Files required for Compilation
- Map the ISRs to their vector locations. For more details refer to section ISR to configure within OS – dependencies
- Compile and build the FEE with all the dependent modules. For more details refer to section Building the Driver

Chapter 10
ISR Reference

None.

10.1 Software specification

The following sections contains driver software specifications.

10.1.1 Define Reference

Constants supported by the FEE driver.

10.1.2 Macros Reference

Macros supported by the FEE driver.

10.1.2.1 Macro FEE_DESERIALIZE

Deserialize scalar parameter from the buffer.

Pre: deserialPtr must be valid pointer.

Post: increments the deserialPtr by sizeof(paramType).

Violates: Function-like macro defined

Prototype: `FEE_DESERIALIZE(deserialPtr,paramVal,paramType)`

Table 10-1. FEE_DESERIALIZE Arguments

Name	Direction	Description
deserialPtr	**input, output**	Pointer to source buffer.
paramVal	**output**	Deserialized parameter.
paramType	**input**	Type of serialized parameter.

10.1.2.2 Macro FEE_SERIALIZE

Serialize scalar parameter into the buffer.

Pre: serialPtr must be valid pointer.

Post: increments the serialPtr by sizeof(paramType).

Violates: Function-like macro defined

Prototype: `FEE_SERIALIZE(paramVal,paramType,serialPtr)`

Table 10-2. FEE_SERIALIZE Arguments

Name	Direction	Description
paramVal	**input**	Serialized parameter.
paramType	**input**	Type of serialized parameter.
serialPtr	**input, output**	Pointer to target buffer.

10.1.3 Enum Reference

Enumeration of all constants supported by the FEE driver.

10.1.4 Function Reference

Functions supported by the FEE driver.

10.1.5 Structs Reference

Data structures supported by the FEE driver.

10.1.6 Types Reference

Types structures supported by the FEE driver.

10.1.7 Variables Reference

Variables supported by the FEE driver.

How to Reach Us:

Home Page:
freescale.com

Web Support:
freescale.com/support

Document Number IM13FEEASR4.0 Rev0003R1.0.1
Revision 1.2

AUTOSAR

freescale™